

Pruebas de Software INF331 | Tarea 3: App. CRUD + pruebas

Estudiante: Javier Torres R. Inicio: 18 junio 2022 Fin: 23 junio 2022

Fecha	18/06	21/06	22/06	22/06	23/06
Versión	0.1 – Estructura documento y supuestos	0.2 – App funcional ver1.0	0.2 – Revisión de pruebas y App ver1.1	1.0 – Borrador video + documento	1.1 – Final AULA

0. [Enunciado](#)
1. [Requerimientos](#)
2. [Supuestos](#)
3. [Entregable](#)
 - 3.1. Enlace proyecto GitHub
 - 3.2. Manual de instalación
 - 3.3. Documento espejo a Video Explicativo
 - 3.3.1. Descripción técnica de la aplicación
 - 3.3.2. Dependencias entre la herramienta y la aplicación
 - 3.3.3. ¿Por qué? y ¿Cómo? se construyeron las pruebas de una determinada manera
 - 3.3.4. Ejecución de las pruebas y reporte de resultados

Construir una aplicación que nos permita registrar información de contacto, mediante un **formulario web**, permitiendo las siguientes acciones:

- Listar contactos (vista por defecto y donde terminan los flujos de otras acciones)
- Agregar contacto
- Editar contacto
- Eliminar contacto

Un contacto tiene los siguientes **campos de información**:

- Nombre
- Primer apellido
- Segundo apellido
- Email
- Teléfono

Construir los **casos de prueba** que nos permitan: Agregar, Editar, Eliminar información por su puesto con la respectiva verificación.

Hacer los **supuestos** que considere **necesario**, pero declararlos por escrito.

Desde el punto de vista gráfico puede **diseñar la aplicación usando sus propios criterios**,

Para realizar las pruebas debe apoyarse en alguno de las herramientas que se listarán a continuación, **debes elegir bien, porque esta elección debe aplicar al alcance solicitado y además la elección puede tener impacto en tu aplicación.**

- **Jest** is a delightful JavaScript Testing Framework with a focus on simplicity. It works with projects using: Babel, TypeScript, Node, React, Angular, Vue and more!
- **Mocha** is a feature-rich JavaScript test framework running on Node.js and in the browser, making asynchronous testing simple and fun. Mocha tests run serially, allowing for flexible and accurate reporting, while mapping uncaught exceptions to the correct test cases.

- **Jasmine** is a behavior-driven development framework for testing JavaScript code. It does not depend on any other JavaScript frameworks. It does not require a DOM. And it has a clean, obvious syntax so that you can easily write tests.
- **Enzyme** is a JavaScript Testing utility for React that makes it easier to test your React Components' output. You can also manipulate, traverse, and in some ways simulate runtime given the output. Enzyme's API is meant to be intuitive and flexible by mimicking jQuery's API for DOM manipulation and traversal.
- **Cypress** is a JavaScript testing automation solution used for web automation. It enables teams to create web test automation scripts. This solution aims to enable frontend developers and test automation engineers to write web tests in the de-facto web language that is JavaScript for web test automation.

1. Requerimientos

Según enunciado se listan los siguientes:

REQ1: Listar contactos (vista por defecto y donde terminan los flujos de otras acciones)

REQ2: Agregar contacto (con campos pertenecientes, 5)

REQ3: Editar contacto (con campos pertenecientes, 5)

REQ4: Eliminar contacto

Además, cada contacto posee la siguiente información:

- Nombre
- Primer apellido
- Segundo apellido
- Email
- Teléfono

2. Supuestos

ID	Descripción	Req. afectados
SP1	Los campos de texto podrán aceptar números	2 – 3
SP2	El nombre y apellidos tendrán un máximo de 30 caracteres y un mínimo de 1	2 – 3
SP3	El email será de la forma: (caracteres)@(caracteres).(caracteres)	2 – 3
SP4	El teléfono será de la forma: +[cód.país] [teléfonoLocal_9dígitos], siendo único para cada contacto	2 – 3
SP5	El listado de contactos no tendrá un orden específico al mostrarse	1
SP6	Al eliminar un contacto no se podrá recuperar	4
SP7	Al incurrir en una acción no válida (falta de campos) o errores en validación se informará con un mensaje, sin guardar contacto, pero sin perder lo escrito	2 – 3
SP8	Se listará a los contactos con todos sus datos visibles a primera vista	1
SP9	Si un contacto (nuevo o edición) entra en conflicto (teléfono) se preguntará si se desea editar y si se cambia el número de teléfono, se tomará en cuenta que es otro contacto.	3

3. Entregable

3.1 Proyecto Github

Enlace GitHub: <https://github.com/J122016/J122016-Tarea-2-INF331-UTFSM>

3.2 Manual de instalación

Debido a la elección del stack de tecnologías para este proyecto (Javascript + html con bootstrap y Jasmine standalone), para ejecutarlo basta con **descargar el repositorio** (.zip o git pull) y luego **abrir** el archivo `ContactManager.html` (aplicativo) en su navegador actual preferido, que acepte la ejecución de javascript. (*probado en Edge y Firefox)

Cabe mencionar que en el aplicativo se encuentra la pestaña **Jasmine tests** donde se importan los tests realizados en `TestRunner.html`, sin embargo, si desea se pueden ver por separado abriendo este último archivo.

3.3 Documento explicativo (espejo video)

3.3.1 Descripción técnica de la App.

Además de los requerimientos y supuestos descritos en secciones 1 y 2 respectivamente, cabe mencionar la elección del stack de tecnologías. Se escogió un stack minimalista (sin framework extras ni backend) ocupando solo HTML con bootstrap + javascript para centrarse en el funcionamiento del framework de pruebas Jasmine. Respecto a este último fue escogido por su independencia (trabajando con su standalone) y por su sintaxis limpia y fácil de comprender para la realización de pruebas unitarias.

3.3.2 Dependencias entre la herramienta y aplicación

Las dependencias del proyecto se organizaron a partir de la plantilla entregada por el standalone de Jasmine, en la cual se distinguen 3 secciones principales:

- Las funciones base `CRUDfunc.js` y la definición de la entidad `contacto.js` pertenecientes a la aplicación, todo esto en la carpeta `src`.
- El o los archivos que poseen las definiciones y casos de prueba, `ContactManagerSpec.js` ubicado en la carpeta `specs`.
- Las vistas de resultados `testrunner.html` junto a la vista del aplicativo `contactManager.html`, ambas ubicadas en la raíz del repositorio.

Con esto para hacer que las pruebas se ejecuten solo tiene que incorporarse el script de las funciones y casos de pruebas a el ejecutor de Jasmine (`testrunner`).

3.3.3 Construcción de pruebas

¿Por qué? - Se construyeron pruebas para mejorar y automatizar el testeo manual normalmente realizado, acelerando este proceso, junto con agregar documentación viva al proyecto.

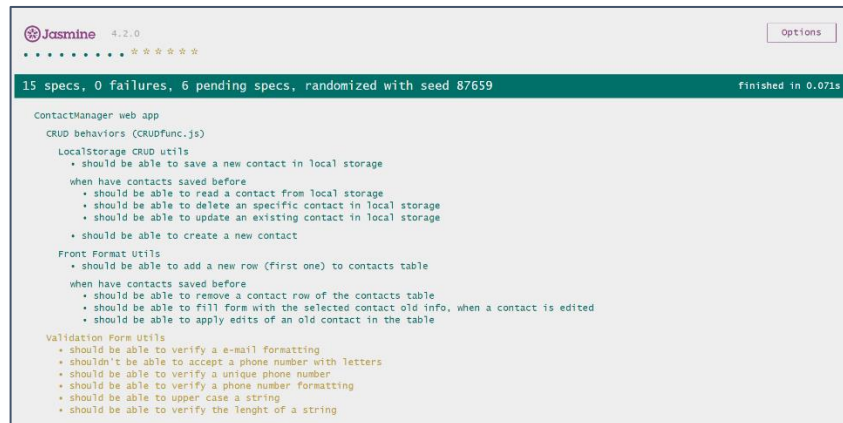
Para que - Para las pruebas se tomaron en consideración los requerimientos CRUD tanto en memoria, localStorage, como de frontend (edición de vistas). Para la validación de los campos se omitieron las pruebas automatizadas debido a que no se tomó como núcleo de la app, junto a realizarse con etiquetas en HTML y clases de Bootstrap ya minimizando los errores de estas.

Cómo - Para la construcción de pruebas se inició con un enfoque BDD para la función crear y guardar del localStorage, es decir, se programó y luego se estableció los casos de prueba y expectativas; esto para comprender el framework de pruebas. Con el resto de los requerimientos se intentó seguir el enfoque TDD donde se describieron los entornos, con la palabra reservada **describe when**, añadiendo acciones previas de prerequisites y posteriores de limpieza con **beforeEach** y **afterEach** respectivamente y con ello los casos de prueba con **it should**, con ello se realizó solo la documentación y firma de las funciones principales de la aplicación.

Así posteriormente se describieron los pasos seguir y expectativas en el archivo de spec pruebas (con la función **expect**). Teniendo ese entorno preliminar, se desarrolló la lógica interna de las funciones para el funcionamiento del aplicativo y se testearon unitariamente con las pruebas realizando otra iteración de desarrollo y limpieza para finalmente integrarlas en la aplicación.

3.3.4 Ejecución de pruebas y resultados

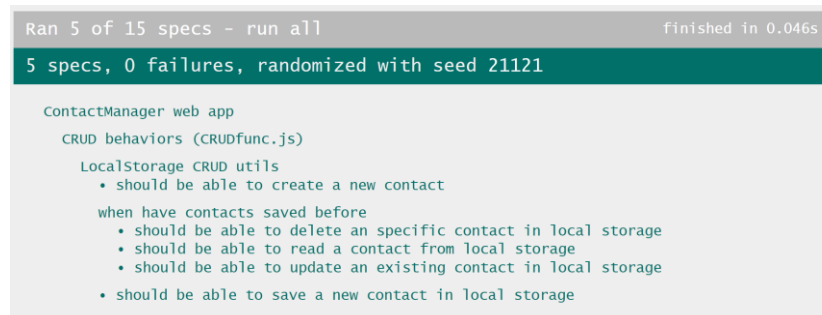
Los resultados, mostrados en testRunner.html. Fueron 15 pruebas escritas, sin embargo, como se dijo anteriormente no se desarrollaron las de validación y se dejaron como pendientes. Todas las demás pasaron exitosamente.



```
Jasmine 4.2.0
*****
15 specs, 0 failures, 6 pending specs, randomized with seed 87659 finished in 0.071s

ContactManager web app
  CRUD behaviors (CRUDfunc.js)
    LocalStorage CRUD utils
      • should be able to save a new contact in local storage
    when have contacts saved before
      • should be able to read a contact from local storage
      • should be able to delete an specific contact in local storage
      • should be able to update an existing contact in local storage
      • should be able to create a new contact
    Front Format utils
      • should be able to add a new row (first one) to contacts table
    when have contacts saved before
      • should be able to remove a contact row of the contacts table
      • should be able to fill form with the selected contact old info, when a contact is edited
      • should be able to apply edits of an old contact in the table
    validation Form utils
      • should be able to verify a e-mail formatting
      • shouldn't be able to accept a phone number with letters
      • should be able to verify a unique phone number
      • should be able to verify a phone number formatting
      • should be able to upper case a string
      • should be able to verify the lenght of a string
```

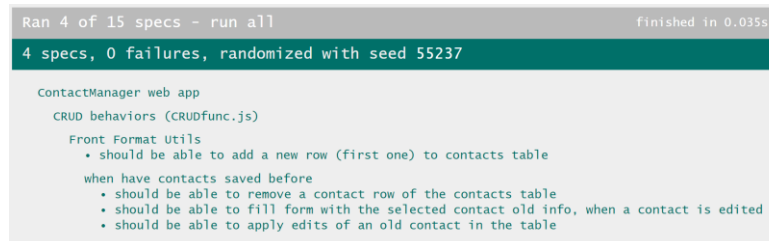
En detalle las pertenecientes al CRUD de integridad de los datos una prueba fue: La aplicación cuando tiene datos guardados, tiene que ser capaz de eliminar un contacto especifico de la memoria, esto se confirmó realizando test donde la expectación de aplicar la función borrar era que al intentar seleccionar los datos estos fueran null.



```
Ran 5 of 15 specs - run all finished in 0.046s
5 specs, 0 failures, randomized with seed 21121

ContactManager web app
  CRUD behaviors (CRUDfunc.js)
    LocalStorage CRUD utils
      • should be able to create a new contact
    when have contacts saved before
      • should be able to delete an specific contact in local storage
      • should be able to read a contact from local storage
      • should be able to update an existing contact in local storage
      • should be able to save a new contact in local storage
```

En el otro aspecto del CRUD, pertenecientes a las vistas, se puede tomar el caso de que, La aplicación, sin ningún requerimiento previo, tiene que ser capaz de agregar la nueva fila de un contacto añadido, esto se logró realizar creando un mock de la tabla contactos, llamando a la función agregar de frontend y revisando que todos los campos fueran agregados con la data correspondiente.



```
Ran 4 of 15 specs - run all finished in 0.035s
4 specs, 0 failures, randomized with seed 55237

ContactManager web app
  CRUD behaviors (CRUDfunc.js)
    Front Format Utils
      • should be able to add a new row (first one) to contacts table
    when have contacts saved before
      • should be able to remove a contact row of the contacts table
      • should be able to fill form with the selected contact old info, when a contact is edited
      • should be able to apply edits of an old contact in the table
```

cabe mencionar que, si bien se probaron los casos de prueba fundamentales para la correcta ejecución de la aplicación, pueden existir muchos otros no tomados en cuenta, por ejemplo, las validaciones, esto debido al acotamiento del testeo y la dificultad de realizar uno que cubra todos los casos posibles, sin embargo, lo realizado es de gran ayuda para la documentación del proyecto y futuras ediciones.